TP2 — Data Communication and Computer Design

Version Control Report

Git & GitHub — Work carried out, evidenced by 28 session screenshots

Student	Mo Anes — GitHub: **anux-10**	**Date**	October 16 2026
Repository	Hello-Repo (anux-10.github.io/Hello-Repo)	**Tools**	Git 2.52.0.windows.1, PowerShell, GitHub, GitHub Pages

1. Objectives of the practical work

The TP aimed at acquiring a complete, professional-grade version-control workflow, both locally (Git) and remotely (GitHub). All steps were documented with screenshots, which are presented in chronological order in the appendix and referenced below.

- Install and verify Git on Windows, and configure the user identity (*git config*).
- Initialise a local repository, stage files, create the initial commit and inspect the history & tags.
- Create, switch, merge and delete branches; understand the Fast-Forward mechanism.
- Create a GitHub repository, connect the local repository to it and push the history.
- Use the GitHub web interface: online commits, remote branches, AI-assisted messages (Copilot) and pull requests.
- Synchronise the local and remote copies with *git pull*, tracking branches and *git push*.
- Publish a website with GitHub Pages served from the main branch.

2. Environment and initial configuration

The work was executed in Windows PowerShell using Git for Windows **2.52.0.windows.1** (Screenshot 1). The global configuration was checked with **git config --list** and **git config --global --list**, establishing the identity **user.name=anux-10** and **user.email=anesmecheri.dev@gmail.com** so that every commit is signed with the right author. Notable settings confirmed: **credential.helper=manager** (GitHub authentication handled by Git Credential Manager), **init.defaultbranch=main**, and **http.sslbackend=schannel** (Screenshots 1–2). A HTML page *index.html* ("Hello World!") served as the sample project (Screenshots 2–4).

3. Creating the local repository and the first commit

git init turned *TP2 DCC* into a Git repository ("Initialized empty Git repository"). **git status** showed *index.html* as an untracked file; **git add .** staged it. An attempt to unstage with **git restore --staged index.html** failed with *fatal: could not resolve HEAD* — normal, since no commit existed yet — and was correctly replaced by **git rm --cached index.html** (Screenshots 2–3). The file was re-staged and the initial commit was created: **git commit -m "initial commit"** → root commit **c806ceb** on **main**, 1 file changed, 12 insertions (Screenshots 4–6).

4. Viewing history and tagging versions

History was inspected with **git log --oneline --graph** (HEAD → main, c806ceb). A lightweight tag **git tag v1.0 c806ceb** marked the first release, and **git show v1.0** displayed the commit details and the diff of *index.html* (Screenshots 7–8). The annotated variant **git tag -a v1.1 c806ceb -m "an update to the commit"** created a richer object storing the tagger, the date and the message (Screenshot 9). The command reference was recalled with **git help** (Screenshot 10).

5. Branching, merging and deleting a branch

A feature branch **hello-world-images** was created and checked out (Screenshot 11). The Hello World page was modified (adding an image to *index.html*) and committed: **git commit -m "Added image to Hello World"** (e4ca801). Switching back to **master** produced an error — *pathspec 'master' did not match any file(s) known to git* — resolved by switching to **main**, the actual default branch. The merge **git merge hello-world-images** was a **Fast-Forward** (Updating c806ceb..e4ca801; 18 insertions, 11 deletions), and **git branch -d hello-world-images** removed the merged branch (Screenshots 11–12).

6. Creating the GitHub repository and pushing the code

A public repository **Hello-Repo** was created on GitHub (owner anux-10, description "new repository for the TP", README + MIT license) (Screenshots 13). Its initial state contains only README.md and LICENSE on the **master** branch (Screenshot 14). The local repository was then connected to GitHub: **git remote add origin https://github.com/anux-10/Hello-Repo.git** followed by **git push origin main**. GitHub returned the classic push report (delta compression, 6 objects, 809 bytes) and created the remote branch: **[new branch] main → main** (Screenshots 15–16). The synchronized repository then showed 5 commits, including a merge of branch 'main' of github.com/anux-11/Hello-Repo (Screenshot 17).

7. Working directly on GitHub: commits, remote branch, Copilot

Two README.md changes were committed online from the GitHub editor ("Update README.md" then "update readme"), committing directly to **master** (Screenshots 18–19). To reproduce the branch-based workflow, a new branch **update-readme-v2** was created from **main** directly on GitHub (Screenshot 20), and the README was edited there with a commit message and description suggested by **GitHub Copilot** ("Update README with additional branch information" / "Clarify the purpose of the repository and update notes.") (Screenshot 21). Back on the local machine, **git pull** fetched the new remote branch (update-readme-v2 → origin/update-readme-v2), **git branch -a** listed it among the remote branches, and **git checkout update-readme-v2** created and switched to a local branch automatically tracking **origin/update-readme-v2** (Screenshots 22–23).

8. Feature work, push and pull request

A further branch **update-html** was created locally; *index.html* was modified, staged and committed ("update index.html file", 30d6f76, 5 insertions), then pushed with **git push origin update-html**, which registered the remote branch and offered to create a pull request (Screenshots 24–25–26–27). A follow-up commit "move to pages" (21 insertions, 18 deletions) finalised the work on the branch (Screenshot 26). On GitHub, the comparison **base: main** ← **compare: update-html** showed that the branches could be merged automatically ("Able to merge") and the pull request was prepared with its title and description (Screenshots 28–29). The repository ended with 3 branches and 7 commits (Screenshots 27–29).

9. Publishing with GitHub Pages

The website was published through **Settings** → **Pages**: source “Deploy from a branch”, branch **main** at **/ (root)**, with HTTPS enforced; the banner “GitHub Pages source saved” confirmed the deployment (Screenshot 30). The final deliverable is reachable at **https://anux-10.github.io/Hello-Repo/** and correctly renders the Hello World page served straight from the repository's main branch (Screenshot 31).

10. Issues encountered and how they were solved

	Cause	Solution
l: could not resolve HEAD	No commit exists yet, so there is no HEAD to compare with.	git rm --cached index.html to unsta
ter' did not match any file(s)	The local default branch is main (init.defaultbranch=main), no master exists.	git checkout main.
ning up-to-date”	The branch and its latest commits had already been pushed to origin after the earlier push.	Checked branch state on GitHub; t

11. Conclusion

The practical work covered the entire Git/GitHub chain: environment configuration, local history (init → add → commit), version marking with lightweight and annotated tags (v1.0, v1.1), parallel development with branches and a Fast-Forward merge, publication of the repository on GitHub with *push*/*pull* synchronisation, online commit and AI-assisted editing (Copilot), remote branch tracking, pull-request preparation, and finally deployment of a live site with GitHub Pages. The 28 screenshots in the appendix document each of these steps

during the session of October 16, 2026.

Deliverable: https://github.com/anux-10/Hello-Repo — live at https://anux-10.github.io/Hello-Repo/

Appendix — Evidence: the 28 session screenshots

Screenshots are ordered chronologically (Windows capture timestamps, October 16 2026, 18:00 → 21:58). Captions link each image to the corresponding step of the report.

Figure 1

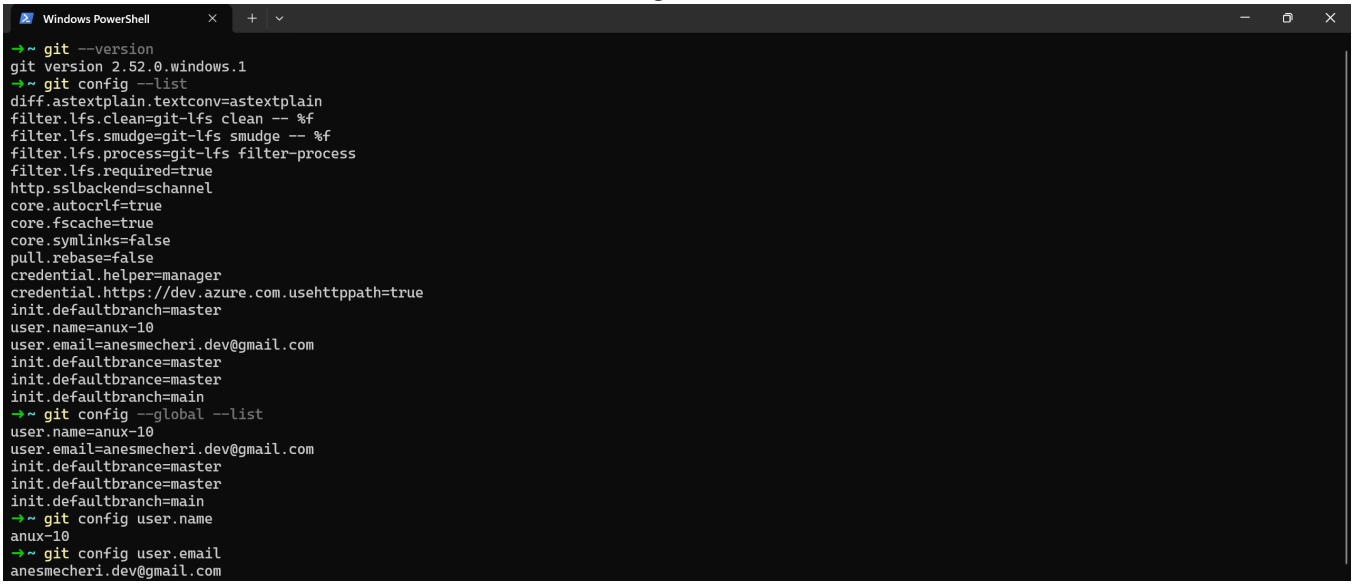


*Figure 1 — Environment verification: **git --version** (Git 2.52.0.windows.1) and **git config --list** / **git config --global --list** confirming the identity (user.name = anux-10, user.email), credential.helper=manager, and init.defaultbranch=main.*

Figure 2

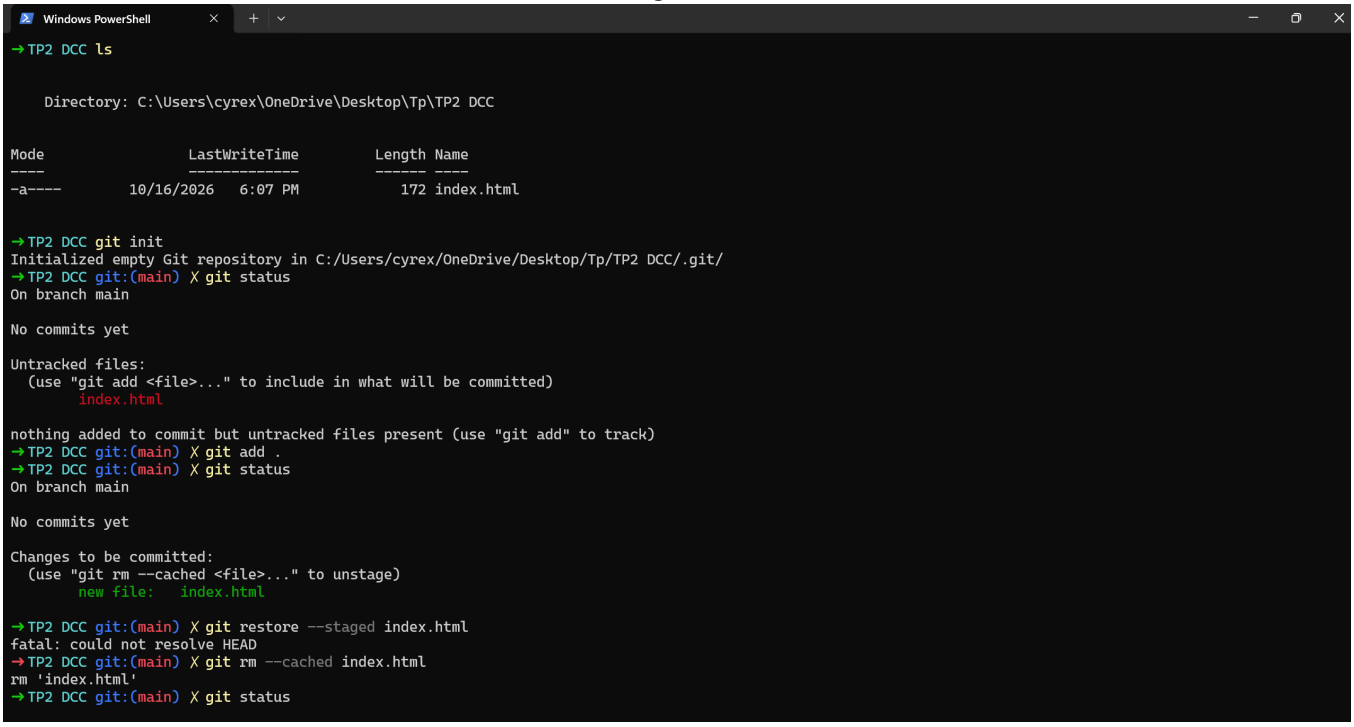


*Figure 2 — Creating the local repository in **TP2 DCC**: **git init**, then **git status** showing the untracked index.html; **git add .** stages it; a failed **git restore --staged** (no commits yet, HEAD unresolved) is replaced by **git rm --cached index.html** to unstage the file.*

Figure 3

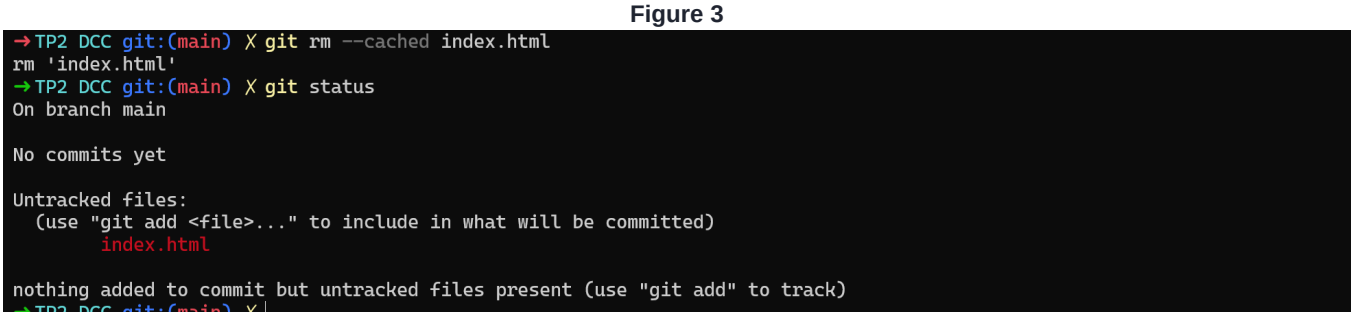


*Figure 3 — After **git rm --cached index.html**: git status confirms the file is back to the untracked state.*

Figure 4

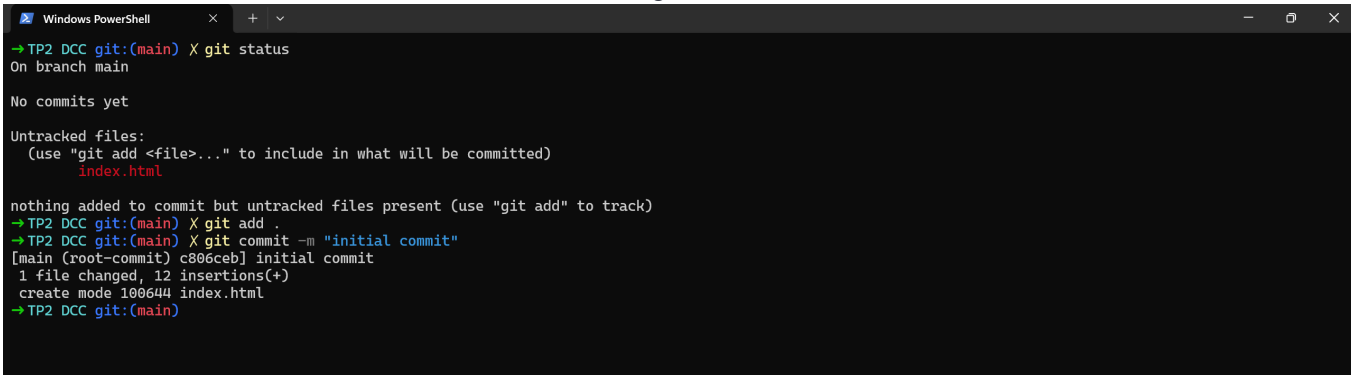


*Figure 4 — Staging and recording history: **git add .** then the first commit **git commit -m "initial commit"** producing the root commit c806ceb (1 file, 12 insertions, create mode 100644 index.html).*

Figure 5

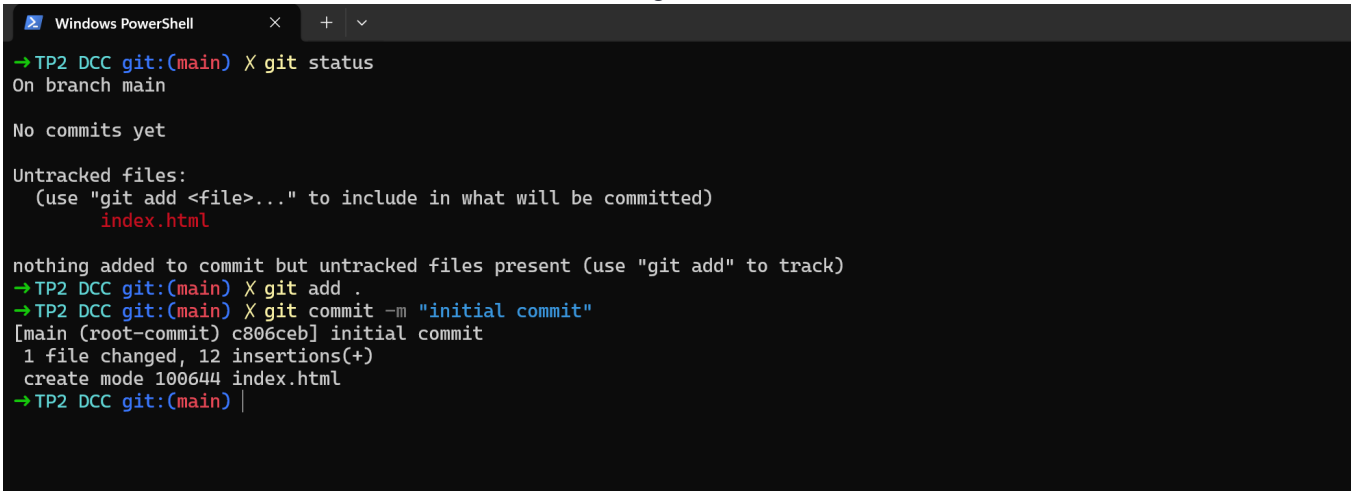


*Figure 5 — Continuation/confirmation of the initiation workflow, ending with the root commit c806ceb on **main**.*

Figure 6



Figure 6 — Local working-tree state after the repository synchronisation with the first commit.

Figure 7

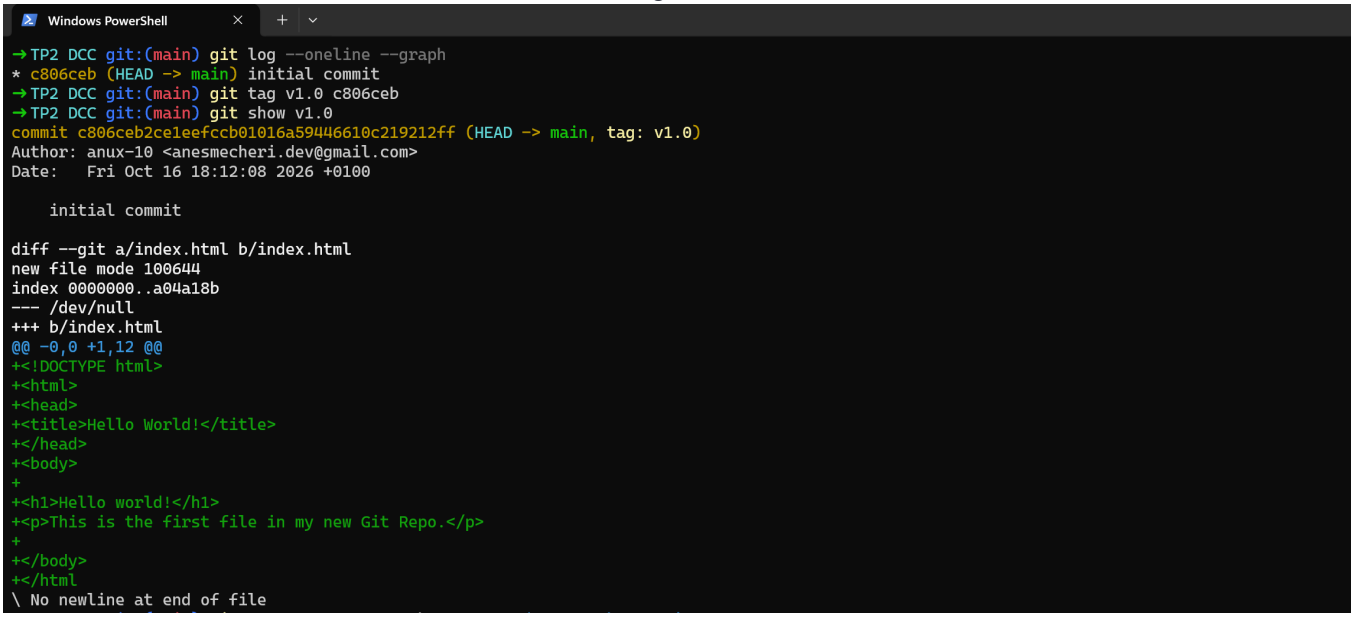


*Figure 7 — Inspecting history and tagging: **git log --oneline --graph**, creating a lightweight tag **git tag v1.0 c806ceb**, and **git show v1.0** displaying the full commit (author anux-10 <anesmecheri.dev@gmail.com>, Fri Oct 16 18:12:08 2026 +0100) together with the diff of index.html (Hello World page).*

Figure 8

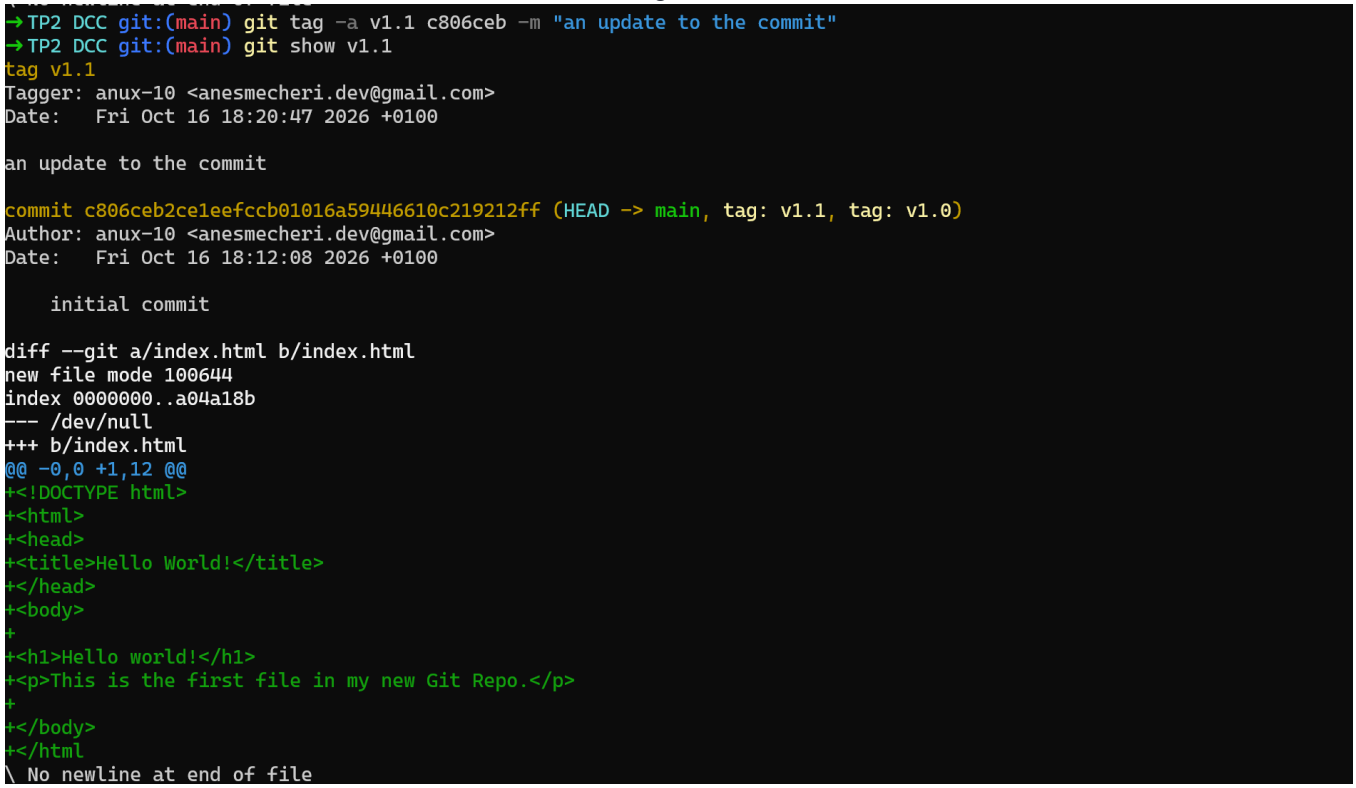


*Figure 8 — Complementary capture of the **git show v1.0** demonstration, confirming the tag points at commit c806ceb on main.*

Figure 9

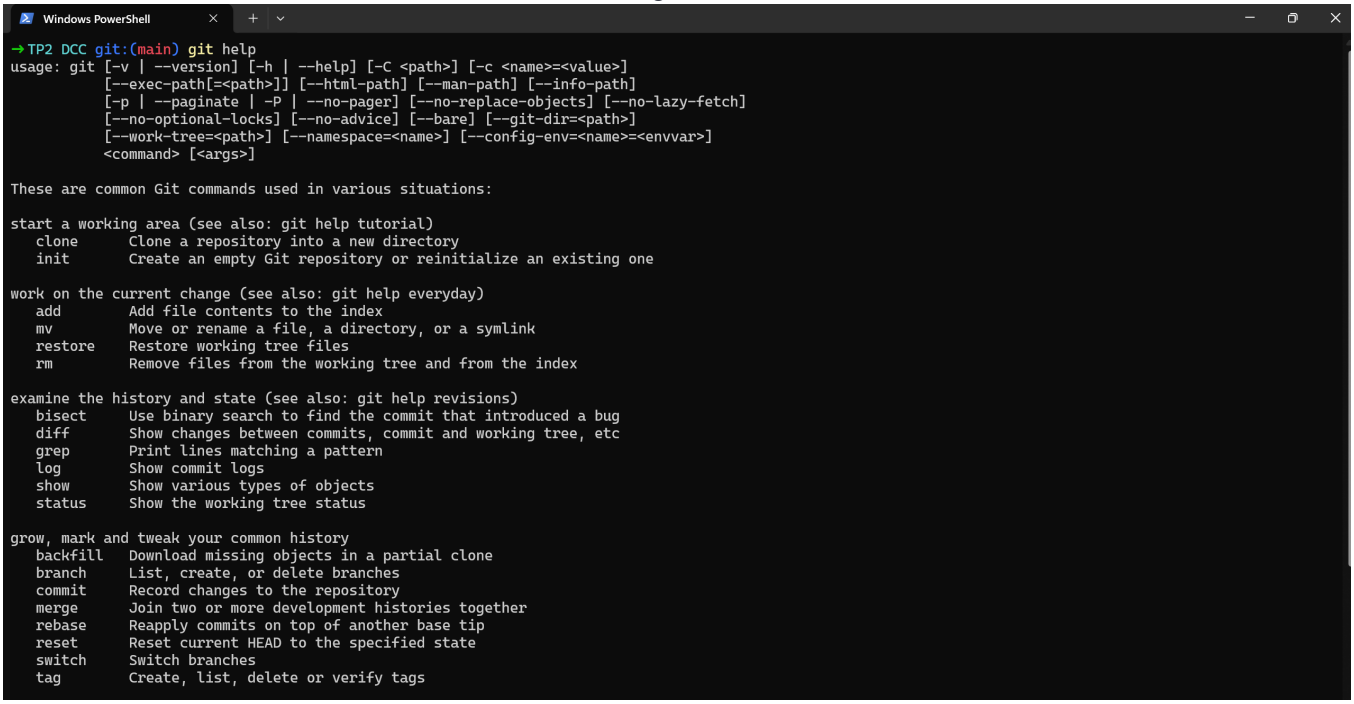


Figure 9 — Lightweight vs annotated tags: ***git tag -a v1.1 c806ceb -m "an update to the commit"****, then* ***git show v1.1*** *which reveals the tagger, tagging date (18:20:47) and the tag message for the same commit.*

Figure 10

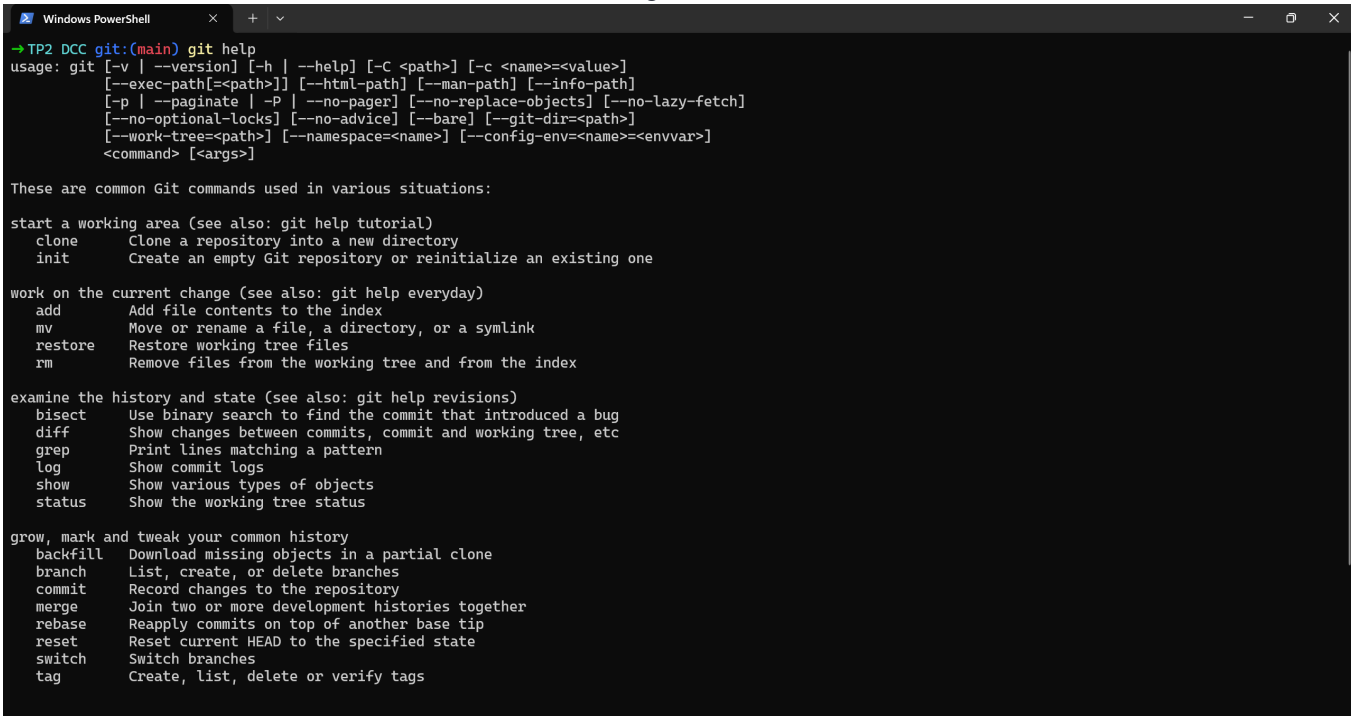


*Figure 10 — **git help**: the official summary of common Git commands (start a working area, work on the current change, examine history and state, grow/mark/tweak history).*

Figure 11

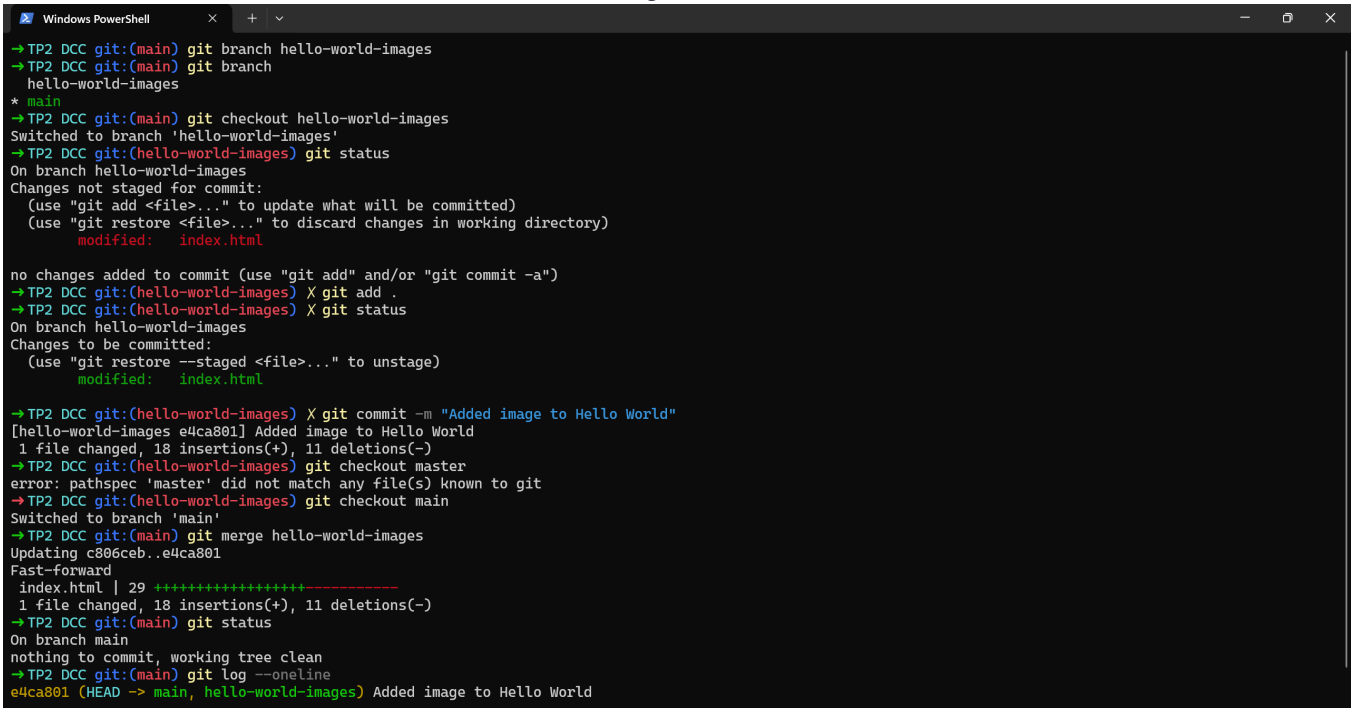


*Figure 11 — Branching and merging: **git branch hello-world-images** and **git checkout** onto it; staging and committing "Added image to Hello World" (commit e4ca801); an error switching to 'master' (branch does not exist) fixed with **git checkout main**; **git merge hello-world-images** performed a Fast-Forward from c806ceb to e4ca801.*

Figure 12

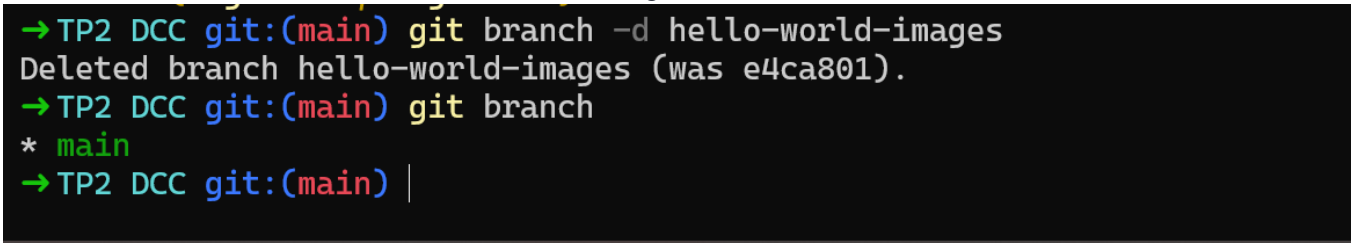


*Figure 12 — Cleaning up: **git branch -d hello-world-images** deletes the merged branch, leaving local **main** up to date.*

Figure 13

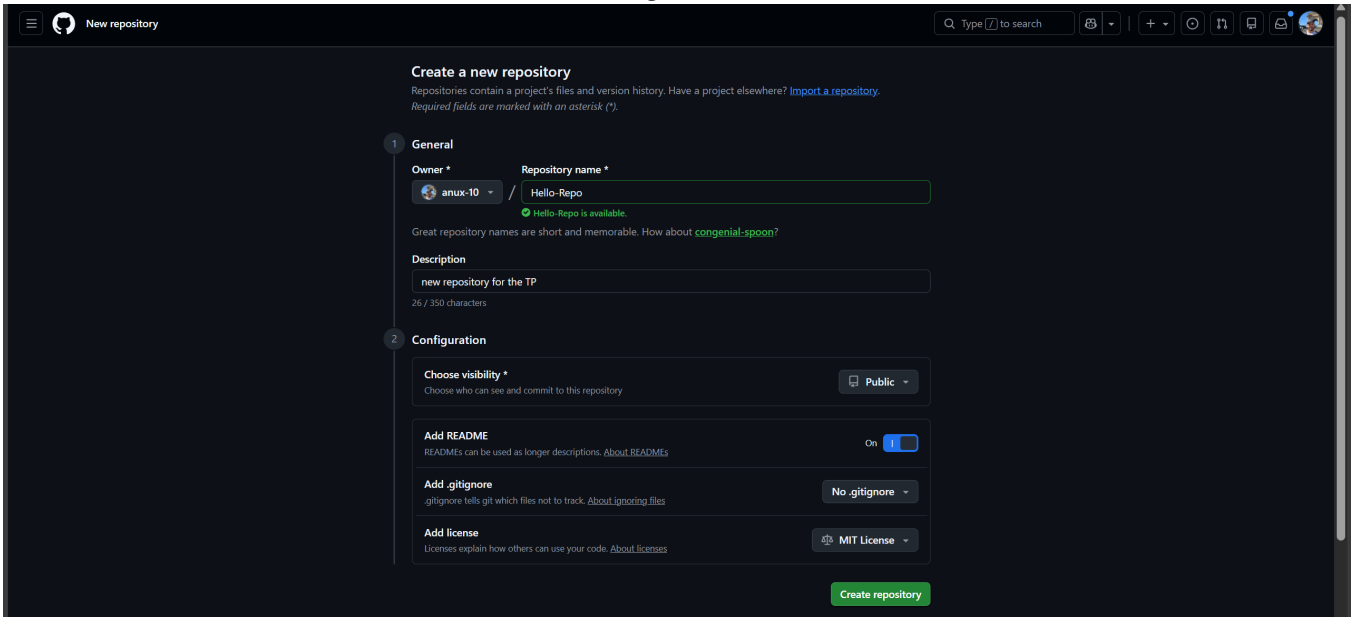


*Figure 13 — GitHub: creating the new public repository **Hello-Repo** (owner anux-10) with description "new repository for the TP", README enabled and MIT license.*

Figure 14

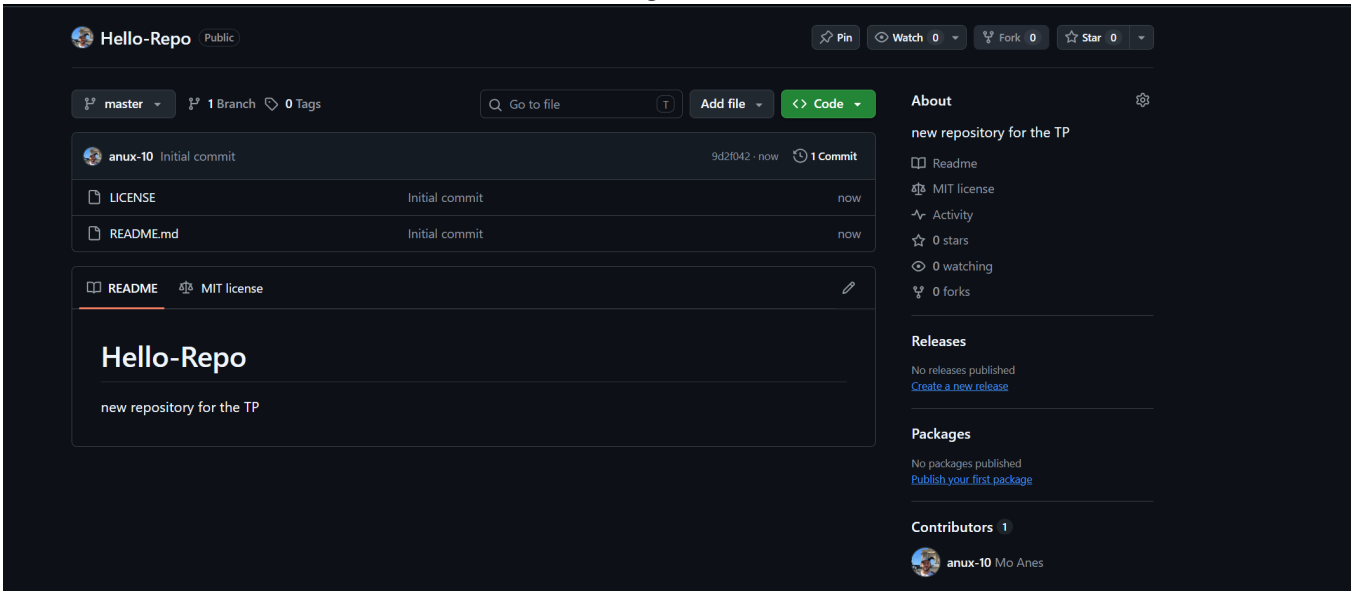


*Figure 14 — The freshly created Hello-Repo page (default branch **master**, README.md + LICENSE from the initial commit) about to receive the local project.*

Figure 15

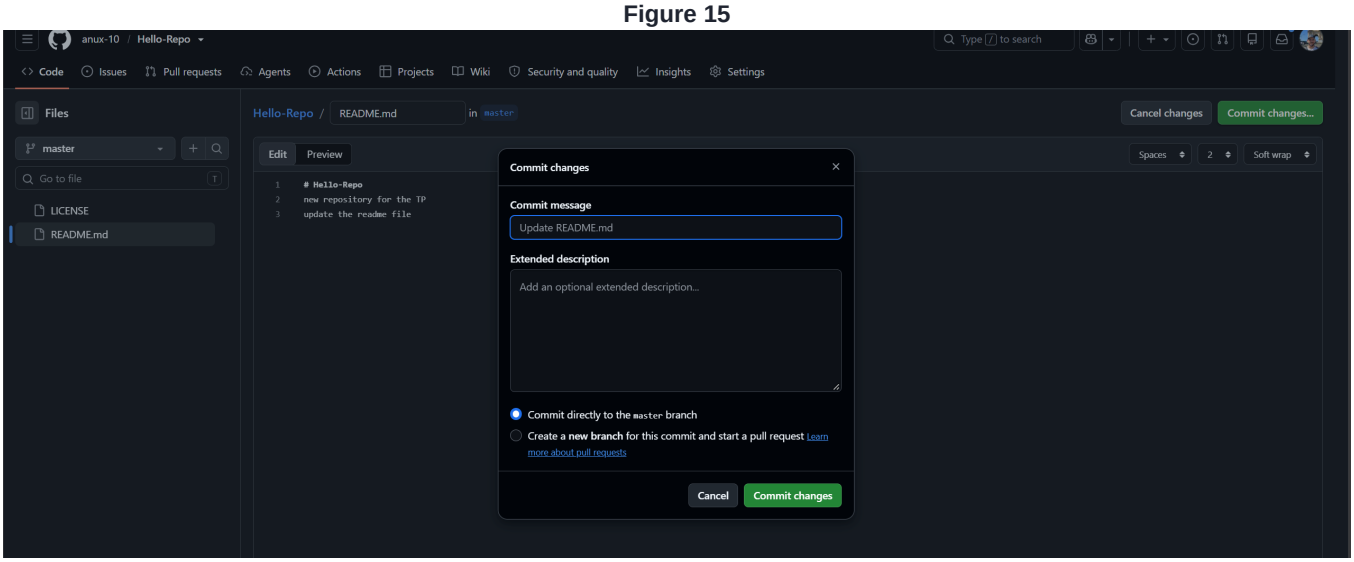


Figure 15 — Editing files directly on GitHub: commit dialog for README.md with the message "Update README.md", committing directly to the master branch.

Figure 16

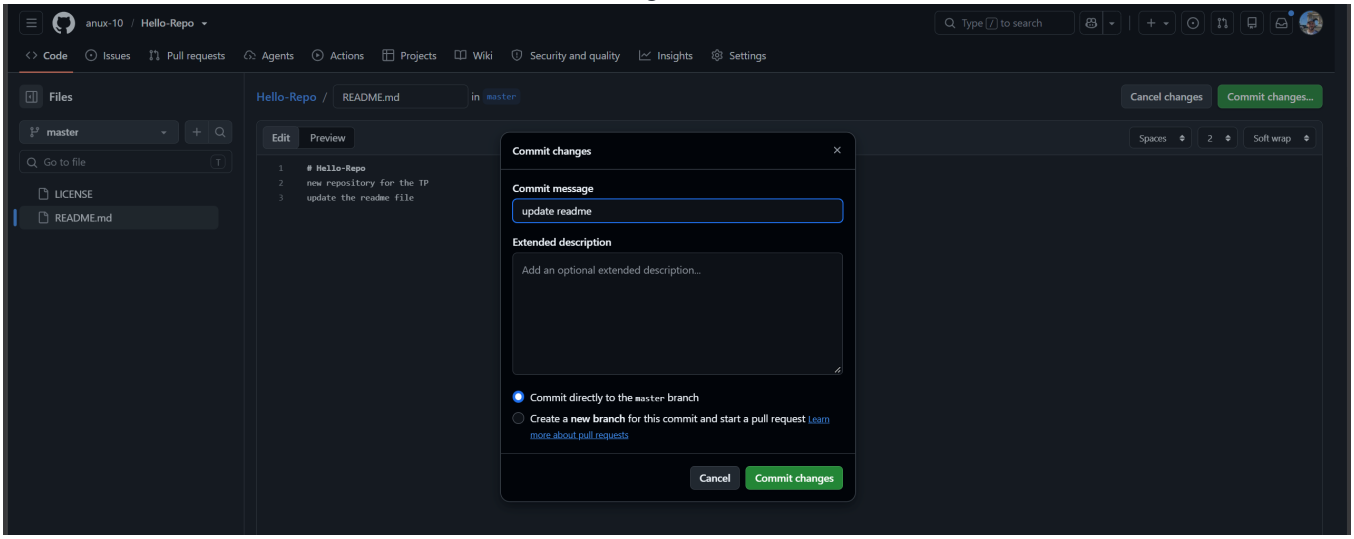


Figure 16 — A second web edit of README.md ("update readme") committed on GitHub, showing the tool's native editing workflow.

Figure 17



Figure 17 — Connecting the local machine to GitHub: ***git remote add origin https://github.com/anux-10/Hello-Repo.git*** *and* ***git push origin main*** *(6 objects, 809 bytes) resulting in* ***[new branch] main -> main****.*

Figure 18

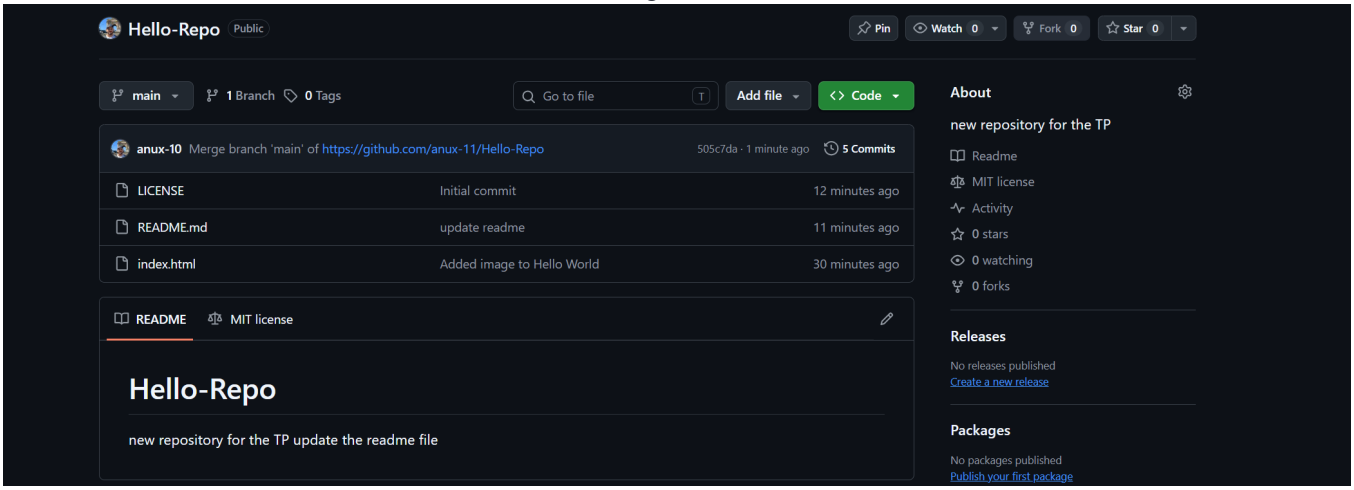


Figure 18 — Hello-Repo on GitHub after the push: main branch containing index.html, LICENSE and README.md (5 commits), including a merge of branch 'main' of github.com/anux-11/Hello-Repo.

Figure 19

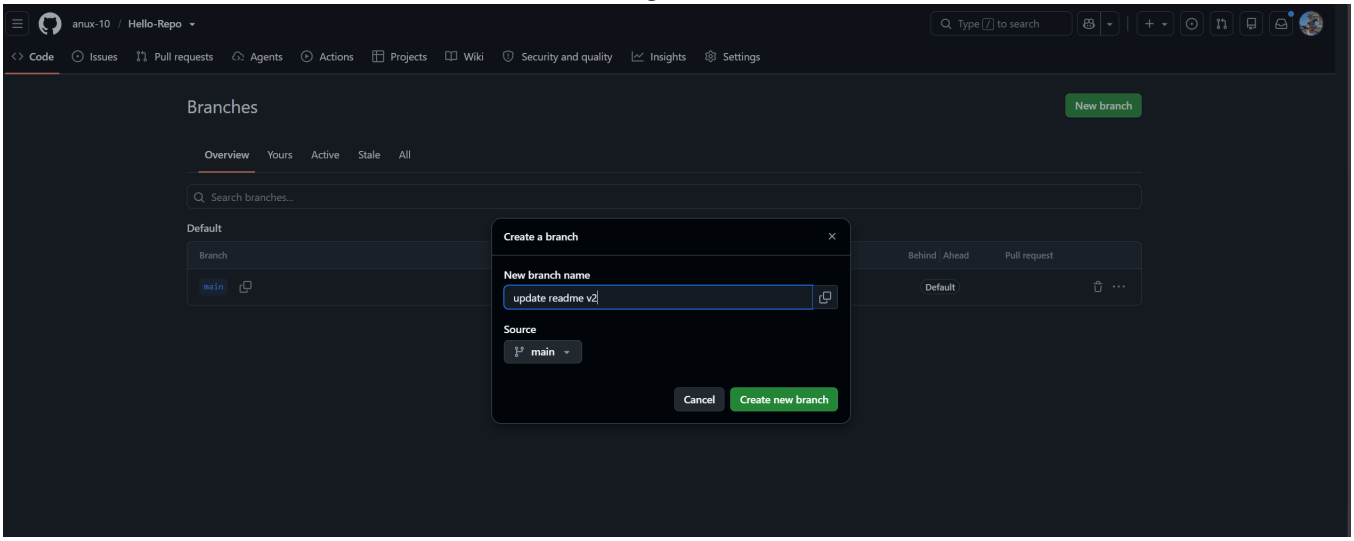


*Figure 19 — Collaboration-style branching on GitHub: creating the remote branch **update-readme-v2** from the source branch **main**.*

Figure 20

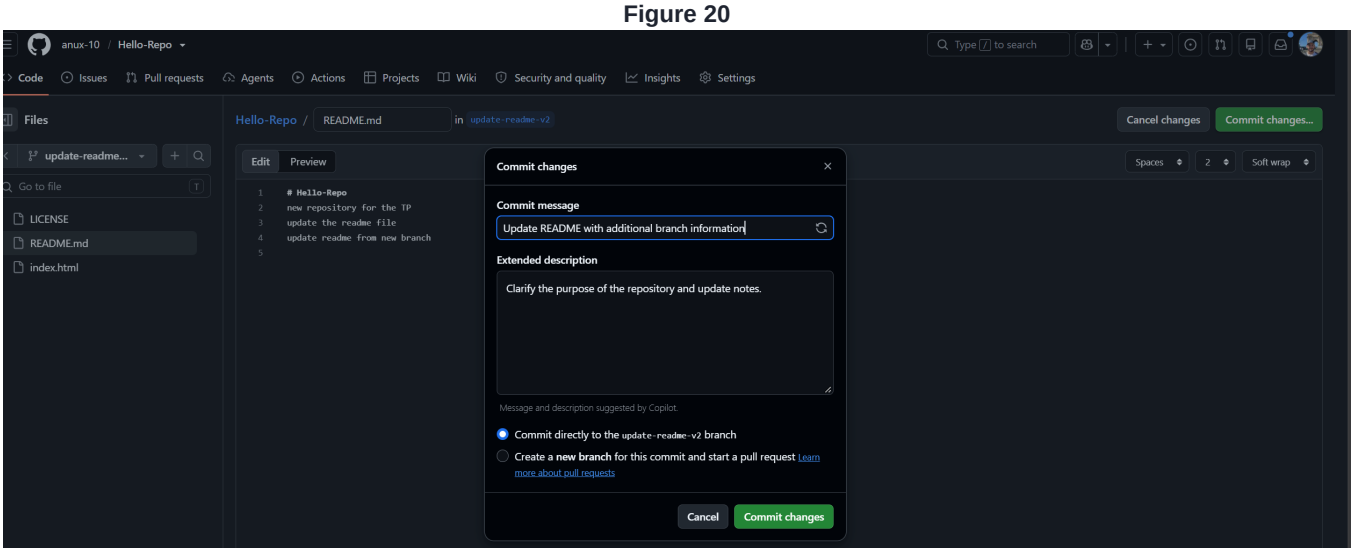


Figure 20 — Editing README.md on GitHub from the update-readme-v2 branch; the commit message and description were suggested by GitHub Copilot, committing directly to the branch.

Figure 21

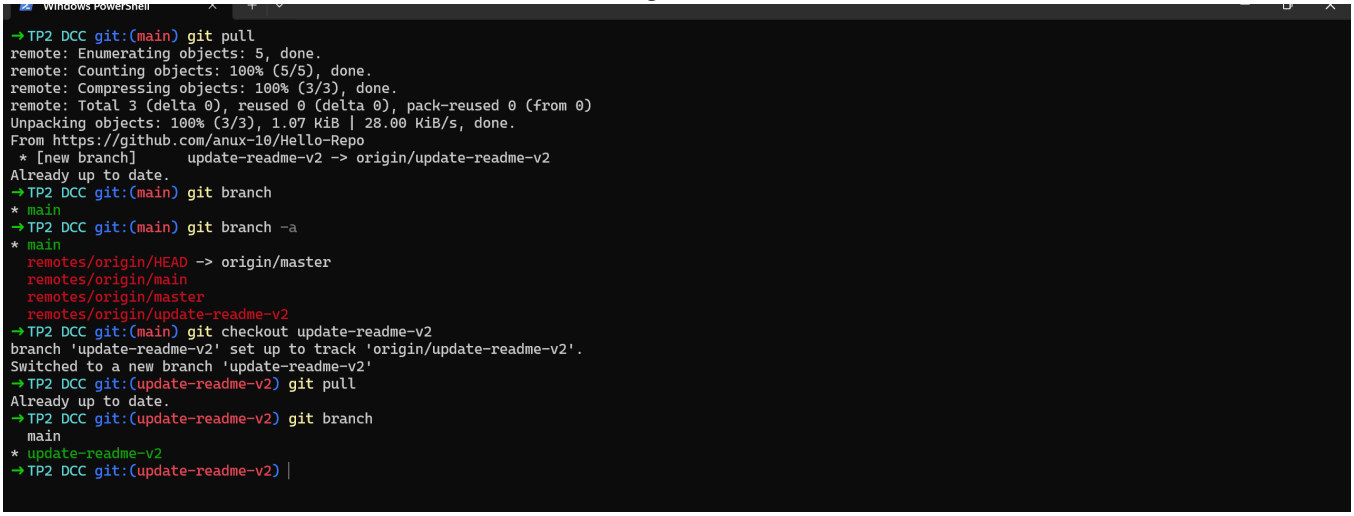


Figure 21 — Synchronising the local clone: ***git pull*** *downloads the new remote branch (update-readme-v2 -> origin/update-readme-v2);* ***git branch -a*** *lists all branches incl. remotes/origin/*;* ***git checkout update-readme-v2*** *creates a local branch tracking origin and switches to it.*

Figure 22

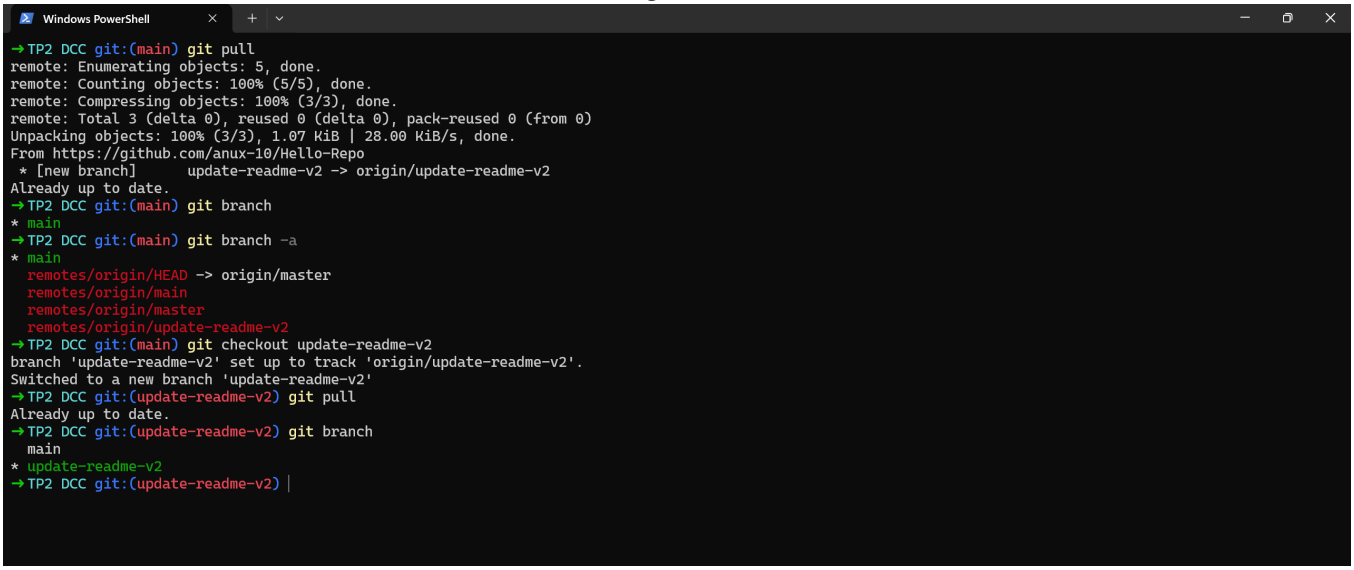


*Figure 22 — As above — second capture confirming **git pull** of origin/update-readme-v2 and the automatic setup of the tracking branch **update-readme-v2**, both locally listed by **git branch**.*

Figure 23

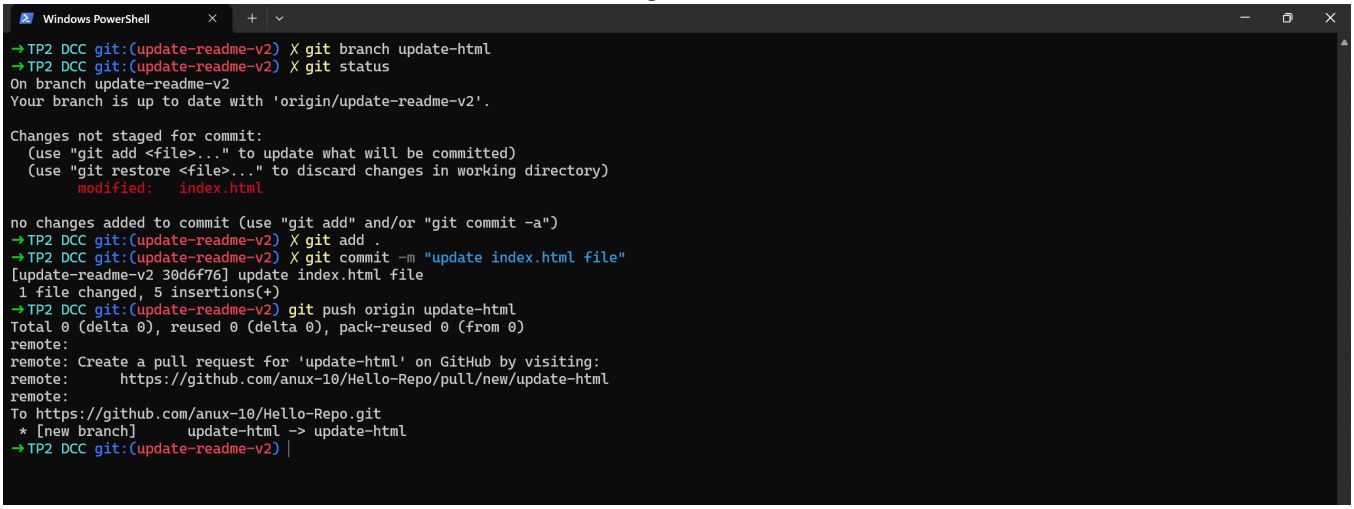


*Figure 23 — Feature work continues locally: **git branch update-html**; index.html modified, staged and committed ("update index.html file", 30d6f76, 5 insertions) on update-readme-v2, then **git push origin update-html** registering the new remote branch.*

Figure 24

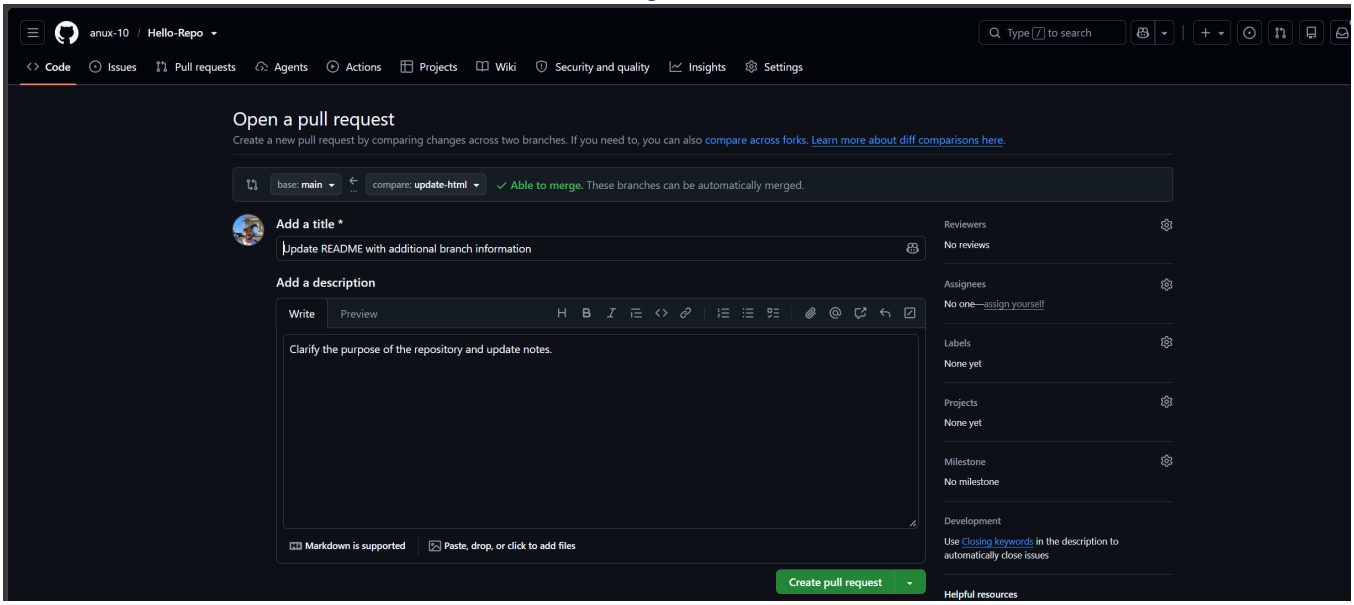


*Figure 24 — Opening a pull request on GitHub: **base: main** ← **compare: update-html**, "Able to merge" banner confirmed, with title "Update README with additional branch information" and a description.*

Figure 25

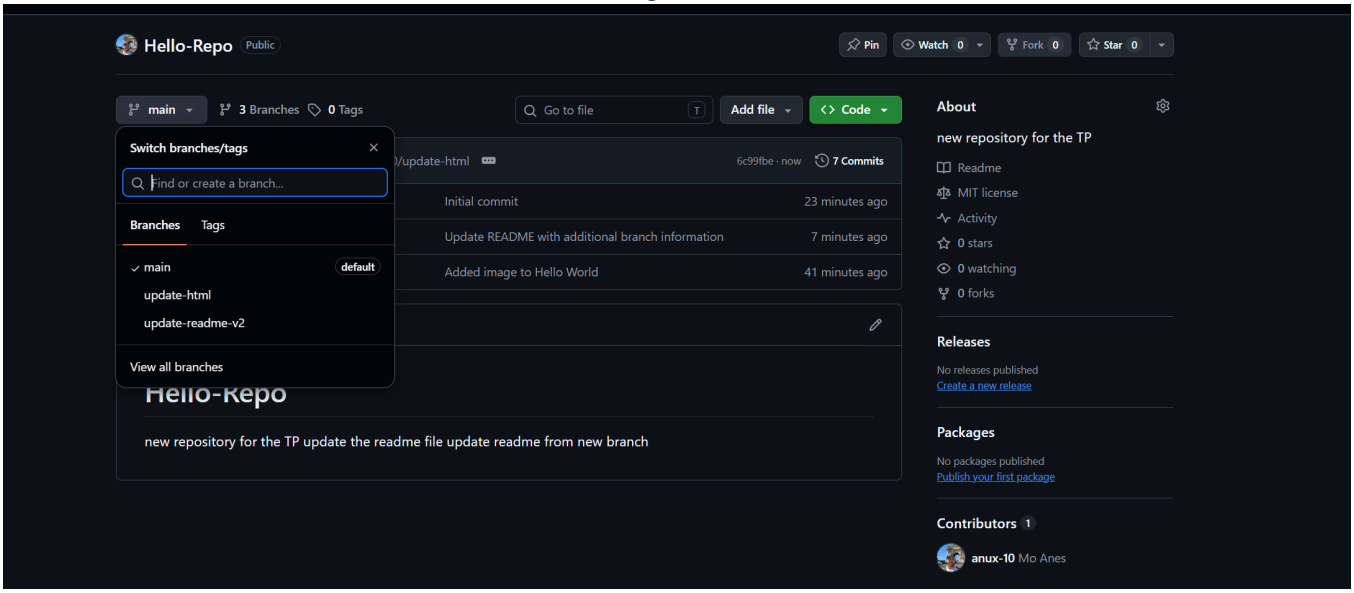


Figure 25 — The repository now shows 3 branches (main, update-html, update-readme-v2) and 7 commits; the branch switcher is shown over the updated Hello-Repo home page.

Figure 26

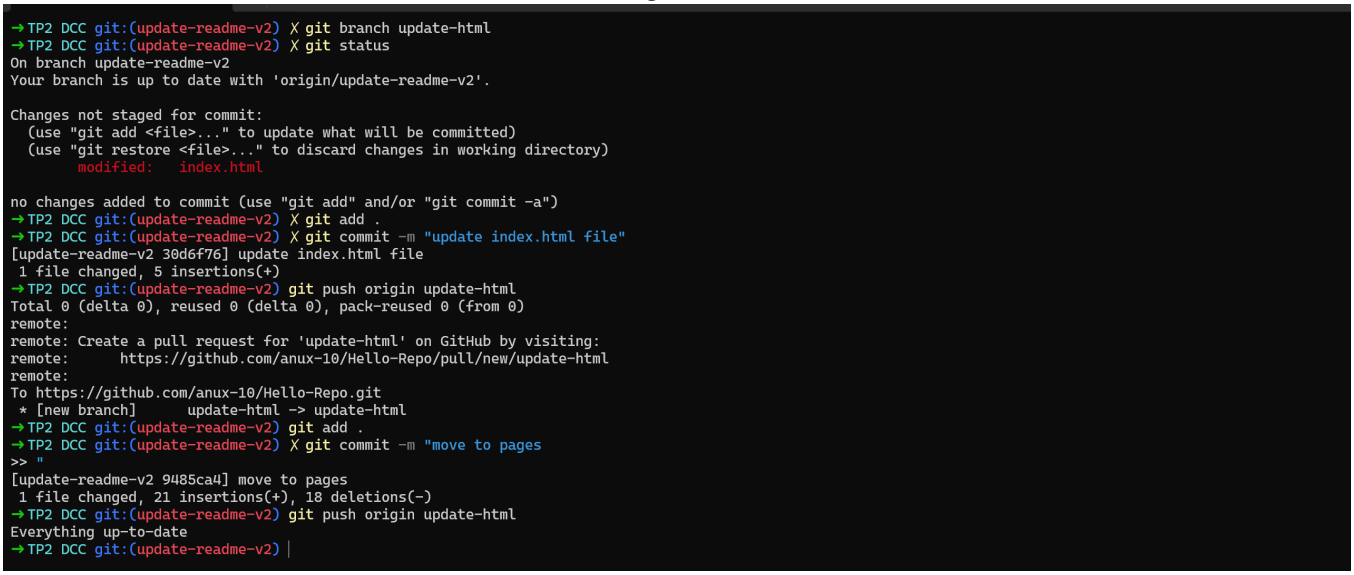


*Figure 26 — Extension of the previous terminal session: a follow-up commit **"move to pages"** (21 insertions, 18 deletions) finalises the branch, then **git push origin update-html** reports "Everything up-to-date".*

Figure 27

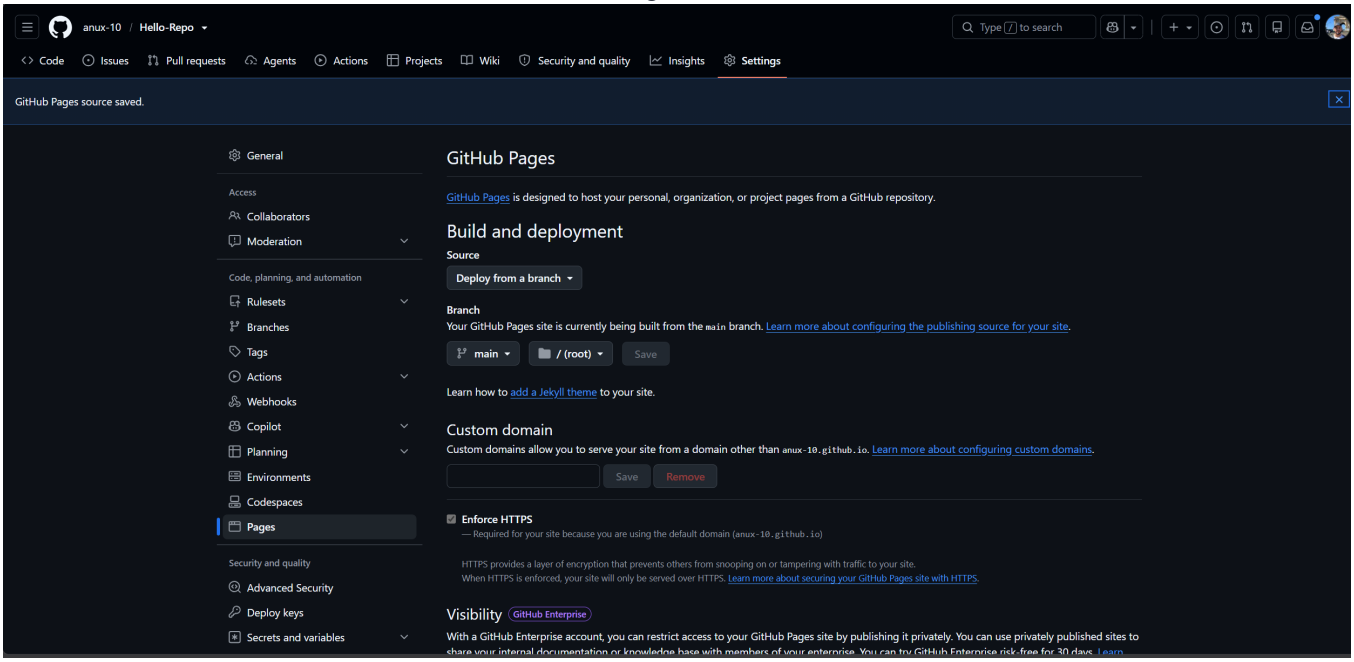


*Figure 27 — Publishing the site: Settings → Pages → Build and deployment from a branch, selecting **main** branch at **/ (root)** with Enforce HTTPS; "GitHub Pages source saved" confirmation is visible.*

Figure 28

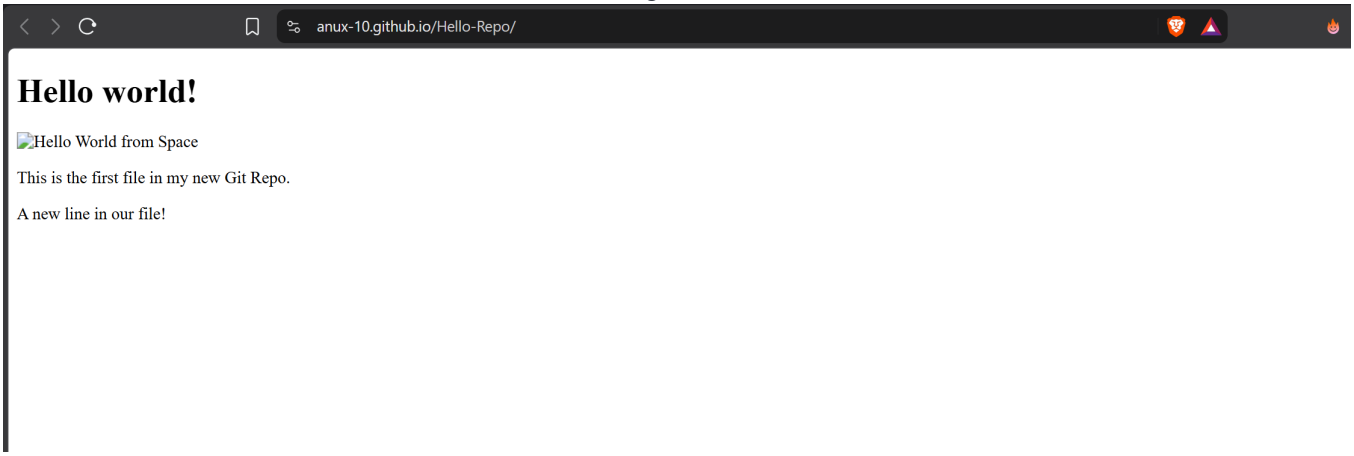


*Figure 28 — Final result: the live website **anux-10.github.io/Hello-Repo/** served by GitHub Pages from the main branch, rendering the Hello World page ("A new line in our file!").*